# Software Requirements Specification

# Payment Ledger Service

## 1. Document Overview

### 1.1 Project Name

**Payment Ledger Service**

### 1.2 Technology Baseline

The service shall be developed using:

- .NET 10
- ASP.NET Core Web API
- C#
- PostgreSQL
- Entity Framework Core
- Docker and Docker Compose
- xUnit / NUnit for testing
- Swagger / OpenAPI for API documentation

### 1.3 Purpose of This Document

This Software Requirements Specification defines the functional, non-functional, architectural, data, API, testing, and scalability requirements for the Payment Ledger Service.

The purpose of this project is to build a production-grade backend service that safely manages wallet balances, money movement, financial transactions, immutable ledger entries, idempotency, and auditability.

This document is intended to guide development, testing, future enhancement, interview explanation, and resume positioning.

---

## 2. Project Summary

### 2.1 Problem Statement

Financial systems must process credits, debits, refunds, and balance updates without duplication, data corruption, or inconsistent ledger state.

A simple wallet application that directly updates balance is not sufficient for production-grade systems because it lacks:

- proper transaction history
- immutable ledger records
- idempotency
- auditability
- concurrency safety
- reconciliation capability
- traceability of every money movement

The Payment Ledger Service solves this by implementing a wallet-ledger backend where every financial operation is recorded as a transaction and posted into immutable ledger entries.

---

## 2.2 Project Objective

The objective is to build a scalable, production-style backend service that supports:

- user creation
- wallet creation
- wallet balance retrieval
- credit operation
- debit operation
- refund operation
- transaction history
- immutable double-entry ledger posting
- idempotency protection
- validation and error handling
- observability
- integration testing
- future extensibility for distributed systems features

---

## 2.3 Existing Reference Project

The previous reference implementation contains:

- wallet-first domain
- users
- wallets
- transactions
- ledger entries
- PostgreSQL
- debit, credit, refund, balance, and transaction history APIs
- Docker-based setup
- basic idempotency using `idempotency_key`

- double-entry ledger entries

The new version shall preserve the same domain intent but improve the implementation using .NET 10, Clean Architecture, stronger idempotency handling, scalable design, better API contracts, better validation, better observability, and richer test coverage.

---

# 3. Scope

## 3.1 In Scope

The first production-grade version shall include:

1. User management
2. Wallet management
3. Wallet balance retrieval
4. Credit wallet
5. Debit wallet
6. Refund transaction
7. Transaction history
8. Transaction details
9. Immutable ledger entries
10. Double-entry posting
11. Idempotency handling
12. Database-level transaction safety
13. Concurrency control
14. Validation
15. Standard API responses
16. Global exception handling
17. Structured logging
18. Correlation ID support
19. Health checks
20. Swagger documentation
21. Dockerized local setup
22. Unit tests
23. Integration tests
24. README documentation

---

## 3.2 Out of Scope for Initial Version

The following features shall not be included in the first version but should be considered for future scalability:

1. Authentication and authorization
2. JWT-based user login
3. Admin dashboard

4. Redis caching
5. Kafka/RabbitMQ event publishing
6. Outbox pattern
7. Multi-currency exchange conversion
8. Partial refunds
9. Scheduled settlement
10. Merchant onboarding
11. KYC verification
12. Webhooks
13. Kubernetes deployment
14. Distributed tracing with OpenTelemetry
15. Real payment gateway integration

---

### 3.3 Future Extension Scope

The architecture shall be designed so that the following can be added later without rewriting the core system:

- Redis-based idempotency cache
- outbox pattern
- event-driven transaction publishing
- admin reconciliation APIs
- authentication
- role-based authorization
- wallet limits
- transaction risk checks
- background jobs
- webhook delivery
- multi-tenant support
- audit log module
- reporting module
- settlement module

---

## 4. Definitions and Terminology

| Term | Meaning |
|---|---|
| User | A customer or account owner in the system |
| Wallet | A financial balance container owned by a user |
| Transaction | A business-level money movement such as credit, debit, or refund |
| Ledger Entry | Immutable accounting record created for each transaction |
| Credit | Operation that increases a wallet balance |

| Term | Meaning |
|---|---|
| Debit | Operation that decreases a wallet balance |
| Refund | Operation that reverses a previous transaction fully or partially |
| Idempotency Key | Client-provided unique key used to safely retry mutation requests |
| Treasury Wallet | Internal platform wallet used as counterparty for wallet credits/debits |
| Double-Entry Ledger | Accounting model where every transaction has matching debit and credit entries |
| Reconciliation | Process of verifying wallet balances against ledger entries |
| Correlation ID | Unique identifier used to trace a request through logs and services |
| ACID | Atomicity, Consistency, Isolation, Durability |
| Immutable Ledger | Ledger records that cannot be edited or deleted after posting |

---

# 5. System Actors

## 5.1 API Client

An external or internal consumer of the API. It may be a frontend app, mobile app, merchant service, or backend service.

Responsibilities:

- create users
- create wallets
- initiate debit/credit/refund requests
- fetch balances
- fetch transaction history

---

## 5.2 System Admin

An internal operator or backend administrator.

Responsibilities:

- view transaction details
- check ledger consistency
- block/unblock wallets
- inspect failed transactions
- perform reconciliation in future versions

### 5.3 Payment Ledger Service

The backend service responsible for:

- validating requests
- executing wallet operations
- updating wallet balances
- creating transaction records
- creating ledger entries
- enforcing idempotency
- maintaining auditability
- exposing APIs

---

### 5.4 Database

PostgreSQL database responsible for:

- storing users
- storing wallets
- storing transactions
- storing ledger entries
- storing idempotency records
- enforcing relational integrity
- supporting transactional consistency

---

## 6. System Architecture

### 6.1 Architecture Style

The system shall follow **Clean Architecture**.

The codebase shall be separated into the following layers:

```
PaymentLedgerService/
│
├── src/
│   ├── PaymentLedger.Api/
│   ├── PaymentLedger.Application/
│   ├── PaymentLedger.Domain/
│   ├── PaymentLedger.Infrastructure/
│   └── PaymentLedger.Shared/
│
```

```
├── tests/
│   ├── PaymentLedger.UnitTests/
│   ├── PaymentLedger.IntegrationTests/
│   └── PaymentLedger.ArchitectureTests/
│
├── docker-compose.yml
├── Dockerfile
├── README.md
└── PaymentLedgerService.sln
```

---

## 6.2 Layer Responsibilities

### 6.2.1 API Layer

Project:

```
PaymentLedger.Api
```

Responsibilities:

- expose HTTP endpoints
- receive requests
- return responses
- configure middleware
- configure Swagger
- configure authentication later
- configure dependency injection
- handle correlation IDs
- map HTTP errors to proper status codes

Should contain:

- controllers or minimal API endpoints
- request models
- response models
- middleware
- filters
- API versioning configuration
- Swagger configuration

---

### 6.2.2 Application Layer

Project:

```
PaymentLedger.Application
```

Responsibilities:

- contain use cases
- orchestrate domain operations
- define service interfaces
- define DTOs
- define validators
- handle idempotency workflow
- coordinate repositories and transactions

Should contain:

- commands
- queries
- handlers/services
- validators
- application interfaces
- DTOs
- application-level exceptions

---

#### 6.2.3 Domain Layer

Project:

```
PaymentLedger.Domain
```

Responsibilities:

- contain core business entities
- contain enums
- contain value objects
- contain domain rules
- contain domain exceptions
- remain independent from infrastructure and framework concerns

Should contain:

- User
- Wallet
- Transaction
- LedgerEntry
- IdempotencyRequest

- Money value object
- TransactionType enum
- TransactionStatus enum
- WalletStatus enum
- LedgerEntryType enum

---

#### 6.2.4 Infrastructure Layer

Project:

```
PaymentLedger.Infrastructure
```

Responsibilities:

- database access
- EF Core DbContext
- repository implementations
- database configurations
- migrations
- external services
- idempotency storage
- transaction manager implementation

Should contain:

- AppDbContext
- EF entity configurations
- repositories
- migrations
- database transaction service
- health check dependencies
- future Redis/Kafka implementations

---

#### 6.2.5 Shared Layer

Project:

```
PaymentLedger.Shared
```

Responsibilities:

- shared response wrappers
- constants

- helper utilities
- error codes
- pagination models

Should contain:

- ApiResponse
- PagedResponse
- ErrorResponse
- ErrorCodes
- DateTimeProvider abstractions
- Result pattern helpers, if used

---

# 7. Functional Requirements

## 7.1 User Module

### FR-USER-001: Create User

The system shall allow creation of a user.

Input:

- full name
- email

Validation:

- full name is required
- email is required
- email must be valid
- email must be unique

Expected result:

- user is created with Active status
- unique user ID is generated
- creation timestamp is stored

---

### FR-USER-002: Get User By ID

The system shall allow retrieving a user by user ID.

Expected result:

- if user exists, return user details
- if user does not exist, return 404 Not Found

---

### FR-USER-003: List Users

The system shall allow listing users with pagination.

Query parameters:

- pageNumber
- pageSize
- searchText
- status

Expected result:

- paginated user list
- total count
- page metadata

---

## 7.2 Wallet Module

### FR-WALLET-001: Create Wallet

The system shall allow creation of a wallet for an existing user.

Input:

- userId
- currency

Validation:

- userId is required
- user must exist
- currency is required
- currency must be valid
- duplicate wallet for same user and same currency may be restricted based on business rule

Expected result:

- wallet is created with zero balance
- wallet status is Active

- wallet ID is generated
- creation timestamp is stored

---

### FR-WALLET-002: Get Wallet Balance

The system shall allow fetching current wallet balance.

Expected response:

- walletId
- userId
- currency
- availableBalance
- walletStatus
- lastUpdatedAt

Rules:

- if wallet does not exist, return 404
- blocked wallet balance can still be viewed
- closed wallet balance can still be viewed

---

### FR-WALLET-003: Get Wallet Details

The system shall allow fetching complete wallet details.

Expected response:

- walletId
- userId
- currency
- balance
- status
- createdAt
- updatedAt

---

### FR-WALLET-004: Block Wallet

The system shall allow blocking a wallet.

Rules:

- blocked wallet cannot perform debit

- blocked wallet cannot perform credit unless admin override is allowed in future
- blocked wallet history remains accessible

---

### FR-WALLET-005: Unblock Wallet

The system shall allow unblocking a wallet.

Rules:

- only blocked wallets can be unblocked
- closed wallets cannot be unblocked

---

### FR-WALLET-006: Close Wallet

The system shall allow closing a wallet.

Rules:

- wallet can be closed only if balance is zero
- closed wallet cannot perform credit, debit, or refund
- closed wallet history remains accessible

---

## 7.3 Credit Module

### FR-CREDIT-001: Credit Wallet

The system shall allow crediting a wallet.

Endpoint behavior:

- increase user wallet balance
- create transaction record
- create ledger entries
- enforce idempotency
- execute all database operations atomically

Input:

- walletId
- amount
- currency
- description
- metadata

- idempotency key

Validation:

- walletId is required
- wallet must exist
- wallet must be Active
- amount must be greater than 0
- currency must match wallet currency
- idempotency key is required
- metadata must be valid JSON

Expected result:

- transaction status becomes Posted
- wallet balance increases
- ledger entries are created
- response returns transaction details

---

### FR-CREDIT-002: Credit Ledger Posting

For every credit operation, the system shall create two ledger entries.

Example:

| Wallet | Entry Type | Amount |
|---|---|---|
| Treasury Wallet | Debit | Amount |
| User Wallet | Credit | Amount |

Rules:

- total debit must equal total credit
- both ledger entries must have the same transaction ID
- ledger entries must be immutable

---

## 7.4 Debit Module

### FR-DEBIT-001: Debit Wallet

The system shall allow debiting a wallet.

Endpoint behavior:

- decrease user wallet balance
- create transaction record
- create ledger entries
- enforce idempotency
- execute all database operations atomically

Input:

- walletId
- amount
- currency
- description
- metadata
- idempotency key

Validation:

- walletId is required
- wallet must exist
- wallet must be Active
- amount must be greater than 0
- currency must match wallet currency
- wallet balance must be sufficient
- idempotency key is required

Expected result:

- transaction status becomes Posted
- wallet balance decreases
- ledger entries are created
- response returns transaction details

---

### FR-DEBIT-002: Insufficient Balance Handling

The system shall reject debit requests when the wallet balance is insufficient.

Expected result:

- no wallet balance change
- no posted ledger entries
- response returns clear business error
- optional failed transaction record may be created depending on implementation decision

Recommended initial behavior:

- do not create ledger entries
- return 400 Bad Request or 409 Conflict with error code `INSUFFICIENT_BALANCE`

---

### FR-DEBIT-003: Debit Ledger Posting

For every debit operation, the system shall create two ledger entries.

Example:

| Wallet | Entry Type | Amount |
|---|---|---|
| User Wallet | Debit | Amount |
| Treasury Wallet | Credit | Amount |

Rules:

- total debit must equal total credit
- both ledger entries must have the same transaction ID
- ledger entries must be immutable

---

## 7.5 Refund Module

### FR-REFUND-001: Refund Transaction

The system shall allow refunding a posted transaction.

Input:

- originalTransactionId
- amount
- currency
- reason
- metadata
- idempotency key

Validation:

- original transaction must exist
- original transaction must be Posted
- original transaction must be refundable
- amount must be greater than 0
- amount must not exceed refundable amount

- currency must match original transaction currency
- idempotency key is required

Initial version rule:

- support full refund first

Future version rule:

- support partial refunds

---

### FR-REFUND-002: Refund Ledger Posting

For refund of a debit transaction, the system shall reverse the original money movement.

Original debit:

| Wallet | Entry Type | Amount |
|---|---|---|
| User Wallet | Debit | Amount |
| Treasury Wallet | Credit | Amount |

Refund:

| Wallet | Entry Type | Amount |
|---|---|---|
| Treasury Wallet | Debit | Amount |
| User Wallet | Credit | Amount |

Rules:

- refund must reference original transaction
- refund must create a new transaction
- original ledger entries must not be modified
- refund ledger entries must be new immutable entries

---

### FR-REFUND-003: Prevent Duplicate Refund

The system shall prevent duplicate refund of the same transaction.

Rules:

- same refund idempotency key should return same refund response
- different idempotency key should still not allow refunding more than refundable amount

- fully refunded transaction should not be refundable again

---

## 7.6 Transaction Module

### FR-TXN-001: Create Transaction Record

Every successful credit, debit, and refund shall create a transaction record.

Transaction fields:

- transactionId
- transactionCode
- type
- status
- sourceWalletId
- destinationWalletId
- amount
- currency
- idempotencyKey
- referenceTransactionId
- description
- metadata
- createdAt
- updatedAt

---

### FR-TXN-002: Get Transaction Details

The system shall allow fetching transaction details by transaction ID.

Expected response:

- transactionId
- transactionCode
- type
- status
- amount
- currency
- sourceWalletId
- destinationWalletId
- referenceTransactionId
- metadata
- createdAt
- ledgerEntries

---

### FR-TXN-003: Get Wallet Transaction History

The system shall allow fetching transactions related to a wallet.

Query parameters:

- pageNumber
- pageSize
- type
- status
- fromDate
- toDate
- sortBy
- sortDirection

Rules:

- response must be paginated
- default sort should be createdAt descending
- wallet must exist

---

### FR-TXN-004: Transaction Status Management

The system shall support the following statuses:

| Status | Meaning |
|---|---|
| Pending | Transaction has started but is not posted yet |
| Posted | Transaction has been successfully posted |
| Failed | Transaction failed and did not affect balance |
| Refunded | Original transaction has been fully refunded |
| PartiallyRefunded | Original transaction has been partially refunded |
| Reversed | Transaction has been reversed by correction flow |

Initial version shall support:

- Posted
- Failed
- Refunded

Future versions may add:

- Pending

- PartiallyRefunded
- Reversed

---

## 7.7 Ledger Module

### FR-LEDGER-001: Create Ledger Entries

The system shall create ledger entries for each posted transaction.

Rules:

- every posted transaction must have corresponding ledger entries
- ledger entries must be append-only
- ledger entries must not be updated after creation
- ledger entries must not be deleted after creation
- ledger entries must store wallet ID, amount, currency, entry type, and transaction ID

---

### FR-LEDGER-002: Double-Entry Validation

The system shall ensure that total debit amount equals total credit amount for each posted transaction.

Example:

```
Total Debit = Total Credit
```

If this rule fails, the transaction shall not be posted.

---

### FR-LEDGER-003: Ledger Immutability

The system shall prevent updates and deletes on ledger entries from application logic.

Rules:

- no update API shall exist for ledger entries
- no delete API shall exist for ledger entries
- mistakes must be corrected through reversal/refund transactions

---

### FR-LEDGER-004: Balance After Transaction

Each ledger entry should optionally store `balanceAfterTransaction`.

Purpose:

- easier debugging
- audit trail
- faster reconciliation
- clear historical balance view

This field is recommended for production-grade design.

---

### FR-LEDGER-005: Ledger Consistency Check

The system shall provide an internal mechanism to check ledger consistency.

Initial version:

- internal service method or test-only verification

Future version:

- admin API for reconciliation

Checks:

- every posted transaction has ledger entries
- debit sum equals credit sum
- wallet stored balance equals calculated ledger balance
- no orphan ledger entries exist

---

## 7.8 Idempotency Module

### FR-IDEMP-001: Require Idempotency Key

The following mutation APIs shall require an idempotency key:

- credit wallet
- debit wallet
- refund transaction

The idempotency key may be supplied through request header:

```
Idempotency-Key: unique-client-key
```

Recommended approach:

- use header-based idempotency key, not request-body idempotency key

---

### FR-IDEMP-002: Store Idempotency Request

The system shall store idempotency request details.

Fields:

- id
- idempotencyKey
- endpoint
- requestHash
- responseBody
- statusCode
- status
- createdAt
- expiresAt

---

### FR-IDEMP-003: Same Key, Same Request

If the same idempotency key is used with the same request payload, the system shall return the previously stored response.

Expected result:

- no duplicate transaction
- no duplicate ledger entry
- same response returned

---

### FR-IDEMP-004: Same Key, Different Request

If the same idempotency key is used with a different request payload, the system shall reject the request.

Expected result:

- return 409 Conflict
- error code: `IDEMPOTENCY_KEY_REUSED_WITH_DIFFERENT_REQUEST`

---

#### FR-IDEMP-005: Idempotency Expiry

Idempotency records shall have expiry.

Recommended initial value:

```
24 hours
```

Future enhancement:

- background cleanup job
- configurable expiry duration

---

## 8. API Requirements

## 8.1 API Versioning

All APIs shall be versioned.

Base path:

```
/api/v1
```

---

## 8.2 Standard Response Format

All successful responses should follow a standard format.

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": {},
  "errors": null,
  "correlationId": "string"
}
```

All error responses should follow a standard format.

```
{
  "success": false,
```

```
  "message": "Validation failed",
  "data": null,
  "errors": [
    {
      "code": "INVALID_AMOUNT",
      "field": "amount",
      "message": "Amount must be greater than zero"
    }
  ],
  "correlationId": "string"
}
```

---

## 8.3 User APIs

### Create User

```
POST /api/v1/users
```

Request:

```
{
  "fullName": "Swagat Mishra",
  "email": "swagat@example.com"
}
```

Response:

```
{
  "success": true,
  "message": "User created successfully",
  "data": {
    "userId": "uuid",
    "fullName": "Swagat Mishra",
    "email": "swagat@example.com",
    "status": "Active",
    "createdAt": "2026-05-30T10:00:00Z"
  },
  "errors": null,
  "correlationId": "string"
}
```

---

### Get User By ID

```
GET /api/v1/users/{userId}
```

---

### List Users

```
GET /api/v1/users?pageNumber=1&pageSize=20&searchText=swagat
```

---

## 8.4 Wallet APIs

### Create Wallet

```
POST /api/v1/wallets
```

Request:

```
{
  "userId": "uuid",
  "currency": "INR"
}
```

---

### Get Wallet Balance

```
GET /api/v1/wallets/{walletId}/balance
```

Response:

```
{
  "success": true,
  "message": "Balance retrieved successfully",
  "data": {
    "walletId": "uuid",
    "userId": "uuid",
    "currency": "INR",
    "availableBalance": 1000.00,
    "status": "Active"
```

```
  },
  "errors": null,
  "correlationId": "string"
}
```

---

### Get Wallet Details

```
GET /api/v1/wallets/{walletId}
```

---

### Block Wallet

```
PATCH /api/v1/wallets/{walletId}/block
```

---

### Unblock Wallet

```
PATCH /api/v1/wallets/{walletId}/unblock
```

---

## 8.5 Money Movement APIs

### Credit Wallet

```
POST /api/v1/wallets/{walletId}/credit
Idempotency-Key: credit-001
```

Request:

```
{
  "amount": 1000.00,
  "currency": "INR",
  "description": "Wallet top-up",
  "metadata": {
    "source": "UPI",
    "externalReference": "UPI123"
```

```
  }
}
```

---

### Debit Wallet

```
POST /api/v1/wallets/{walletId}/debit
Idempotency-Key: debit-001
```

Request:

```
{
  "amount": 250.00,
  "currency": "INR",
  "description": "Merchant payment",
  "metadata": {
    "merchantId": "M123",
    "orderId": "ORD789"
  }
}
```

---

### Refund Transaction

```
POST /api/v1/transactions/{transactionId}/refund
Idempotency-Key: refund-001
```

Request:

```
{
  "amount": 250.00,
  "currency": "INR",
  "reason": "Customer cancelled order",
  "metadata": {
    "supportTicketId": "SUP-42"
  }
}
```

---

## 8.6 Transaction APIs

### Get Transaction Details

```
GET /api/v1/transactions/{transactionId}
```

---

### Get Wallet Transactions

```
GET /api/v1/wallets/{walletId}/transactions?
pageNumber=1&pageSize=20&type=Debit&status=Posted
```

---

## 8.7 Health Check APIs

### Basic Health Check

```
GET /health
```

Expected response:

```
{
  "status": "Healthy"
}
```

---

### Detailed Health Check

```
GET /health/details
```

Expected checks:

- API running
- PostgreSQL reachable
- database migration status
- optional future Redis status
- optional future message broker status

---

# 9. Database Requirements

## 9.1 Database

The system shall use PostgreSQL as the primary database.

---

## 9.2 Core Tables

The initial version shall include the following tables:

1. users
2. wallets
3. transactions
4. ledger_entries
5. idempotency_requests

---

## 9.3 Users Table

```
users
-----
user_id UUID PRIMARY KEY
full_name VARCHAR(200) NOT NULL
email VARCHAR(320) NOT NULL UNIQUE
status VARCHAR(30) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NULL
```

---

## 9.4 Wallets Table

```
wallets
-------
wallet_id UUID PRIMARY KEY
user_id UUID NOT NULL
currency VARCHAR(3) NOT NULL
balance NUMERIC(18,2) NOT NULL
status VARCHAR(30) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NULL
row_version BYTEA / concurrency token
```

Constraints:

- balance >= 0
- currency required
- user_id foreign key to users

Indexes:

- user_id
- user_id + currency
- status

---

## 9.5 Transactions Table

```
transactions
------------
transaction_id UUID PRIMARY KEY
transaction_code VARCHAR(50) NOT NULL UNIQUE
type VARCHAR(30) NOT NULL
status VARCHAR(30) NOT NULL
source_wallet_id UUID NOT NULL
destination_wallet_id UUID NOT NULL
amount NUMERIC(18,2) NOT NULL
currency VARCHAR(3) NOT NULL
idempotency_key VARCHAR(200) NOT NULL
reference_transaction_id UUID NULL
description VARCHAR(500) NULL
failure_reason VARCHAR(1000) NULL
metadata JSONB NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NULL
```

Constraints:

- amount > 0
- source_wallet_id foreign key to wallets
- destination_wallet_id foreign key to wallets
- reference_transaction_id foreign key to transactions
- idempotency_key unique for mutation operation context

Indexes:

- transaction_code
- idempotency_key
- source_wallet_id + created_at

- destination_wallet_id + created_at
- reference_transaction_id
- status
- type
- created_at

---

## 9.6 Ledger Entries Table

```
ledger_entries
--------------
ledger_entry_id BIGSERIAL PRIMARY KEY
transaction_id UUID NOT NULL
wallet_id UUID NOT NULL
entry_type VARCHAR(20) NOT NULL
amount NUMERIC(18,2) NOT NULL
currency VARCHAR(3) NOT NULL
balance_after_transaction NUMERIC(18,2) NULL
created_at TIMESTAMPTZ NOT NULL
```

Constraints:

- amount > 0
- transaction_id foreign key to transactions
- wallet_id foreign key to wallets
- entry_type must be Debit or Credit

Indexes:

- transaction_id
- wallet_id + created_at
- entry_type

---

## 9.7 Idempotency Requests Table

```
idempotency_requests
--------------------
id UUID PRIMARY KEY
idempotency_key VARCHAR(200) NOT NULL
endpoint VARCHAR(300) NOT NULL
request_hash VARCHAR(128) NOT NULL
response_body JSONB NULL
status_code INT NULL
status VARCHAR(30) NOT NULL
```

```
created_at TIMESTAMPTZ NOT NULL
expires_at TIMESTAMPTZ NOT NULL
```

Constraints:

- unique idempotency_key + endpoint

Indexes:

- idempotency_key + endpoint
- expires_at
- status

---

# 10. Business Rules

## 10.1 Money Handling Rules

1. The system shall use `decimal` in C# for money values.
2. The database shall use `NUMERIC(18,2)` or stricter precision based on final design.
3. Floating-point types such as `float` and `double` shall not be used for money.
4. Amount must always be greater than zero.
5. Currency must be explicitly supplied for financial operations.
6. Currency must match wallet currency.
7. Negative wallet balance shall not be allowed in initial version.

---

## 10.2 Wallet Rules

1. Wallet must belong to a valid user.
2. Wallet is created with zero balance.
3. Wallet status must be Active before debit.
4. Wallet status must be Active before credit.
5. Wallet cannot be closed if balance is greater than zero.
6. Wallet balance must not become negative.
7. Wallet currency cannot be changed after creation.

---

## 10.3 Transaction Rules

1. Every posted credit, debit, and refund shall create one transaction record.
2. Every posted transaction shall create ledger entries.
3. Failed transaction shall not affect wallet balance.
4. Refund must reference original transaction.
5. Refund must not exceed original transaction amount.

6. Transaction and ledger entries must be persisted atomically.
7. Transaction status must represent actual processing outcome.

---

## 10.4 Ledger Rules

1. Ledger entries must be immutable.
2. Ledger entries must be append-only.
3. Ledger entries must not be updated.
4. Ledger entries must not be deleted.
5. Every posted transaction must have balanced debit and credit entries.
6. Ledger entries must use the same currency as the transaction.
7. Ledger entries must reference a valid transaction.
8. Ledger entries must reference a valid wallet.

---

## 10.5 Idempotency Rules

1. Mutation APIs must require idempotency key.
2. Same idempotency key with same request must return original response.
3. Same idempotency key with different request must return conflict.
4. Idempotency records must expire after configured duration.
5. Idempotency processing must be concurrency-safe.

---

# 11. Non-Functional Requirements

## 11.1 Scalability

The system shall be designed to scale horizontally.

Requirements:

1. API must be stateless.
2. Session state must not be stored in application memory.
3. Database should be the source of truth.
4. Idempotency should initially use database storage.
5. Future Redis support should be possible.
6. Future message broker support should be possible.
7. Application services should depend on interfaces, not concrete infrastructure.
8. Write operations must be isolated and transaction-safe.
9. Query operations should support pagination.
10. Indexes must be created for common query patterns.

---

## 11.2 Maintainability

Requirements:

1. Code shall follow Clean Architecture.
2. Domain logic should not depend on ASP.NET Core.
3. Application logic should be testable without HTTP layer.
4. Infrastructure implementation should be replaceable.
5. Naming should be clear and business-oriented.
6. Project should avoid unnecessary overengineering in version 1.
7. Public methods should have clear responsibility.
8. Validation logic should be centralized using validators.
9. Error handling should be consistent.
10. README should explain setup, architecture, and API usage.

---

## 11.3 Reliability

Requirements:

1. Wallet mutation operations must be atomic.
2. Database transactions must be used for credit, debit, and refund.
3. Partial writes must not be allowed.
4. Duplicate processing must be prevented through idempotency.
5. Concurrency conflicts must be handled.
6. Failed operations must return clear errors.
7. Logs must contain correlation ID.
8. Health checks must expose service status.
9. Integration tests must cover critical financial flows.

---

## 11.4 Performance

Initial performance requirements:

1. Balance lookup should complete quickly through stored wallet balance.
2. Transaction history must be paginated.
3. Common lookup fields must be indexed.
4. Large metadata should not be used for primary filtering.
5. API response payload should be controlled.
6. N+1 database queries should be avoided.
7. Read-only queries should use no-tracking EF Core queries.

---

## 11.5 Security

Initial version security requirements:

1. Validate all input.
2. Do not expose stack traces in API responses.
3. Do not log sensitive request data.
4. Use parameterized queries through EF Core.
5. Restrict CORS in non-development environments.
6. Store secrets in environment variables.
7. Do not commit database passwords or secrets.
8. Use HTTPS in production.

Future security requirements:

1. JWT authentication
2. Role-based authorization
3. Admin-only APIs
4. API keys for service-to-service communication
5. rate limiting
6. audit logs
7. request signing for critical payment APIs

---

## 11.6 Observability

Requirements:

1. Structured logging shall be implemented.
2. Every request shall have correlation ID.
3. Logs shall include request path, status code, duration, and correlation ID.
4. Health check endpoint shall be available.
5. Database health shall be checked.
6. Critical business failures shall be logged.
7. Future OpenTelemetry integration shall be possible.

Recommended logging library:

```
Serilog
```

---

## 11.7 Testability

Requirements:

1. Unit tests shall cover domain rules.
2. Unit tests shall cover application services.
3. Integration tests shall cover API and database behavior.
4. Idempotency tests shall be included.
5. Concurrency tests shall be included.
6. Refund tests shall be included.
7. Insufficient balance tests shall be included.
8. Ledger consistency tests shall be included.

---

# 12. Concurrency Requirements

### 12.1 Wallet Balance Concurrency

The system shall prevent race conditions during concurrent debit and credit operations.

Example risk:

- wallet balance is 100
- two debit requests of 80 happen at same time
- both read balance as 100
- both succeed incorrectly

Required behavior:

- only one debit should succeed
- second debit should fail due to insufficient balance or concurrency conflict

Implementation options:

1. database transaction with row-level locking
2. optimistic concurrency token
3. PostgreSQL `SELECT ... FOR UPDATE`
4. atomic update with balance condition

Recommended initial approach:

- use database transaction
- lock wallet row before balance update
- verify balance inside transaction
- update balance
- create transaction and ledger entries
- commit

## 13. Error Handling Requirements

### 13.1 Error Response Standard

All errors shall follow a consistent format.

```
{
  "success": false,
  "message": "Error message",
  "data": null,
  "errors": [
    {
      "code": "ERROR_CODE",
      "field": "fieldName",
      "message": "Detailed error message"
    }
  ],
  "correlationId": "string"
}
```

### 13.2 Error Codes

The system shall use standard error codes.

| Error Code | Meaning |
|---|---|
| USER_NOT_FOUND | User does not exist |
| WALLET_NOT_FOUND | Wallet does not exist |
| WALLET_NOT_ACTIVE | Wallet is not active |
| INVALID_AMOUNT | Amount is invalid |
| INVALID_CURRENCY | Currency is invalid |
| CURRENCY_MISMATCH | Request currency does not match wallet currency |
| INSUFFICIENT_BALANCE | Wallet balance is insufficient |
| TRANSACTION_NOT_FOUND | Transaction does not exist |
| TRANSACTION_NOT_REFUNDABLE | Transaction cannot be refunded |
| REFUND_AMOUNT_EXCEEDED | Refund amount exceeds refundable amount |
| IDEMPOTENCY_KEY_REQUIRED | Idempotency key is missing |

| Error Code | Meaning |
|---|---|
| IDEMPOTENCY_CONFLICT | Same key used for different request |
| VALIDATION_FAILED | Request validation failed |
| CONCURRENCY_CONFLICT | Concurrent update conflict occurred |
| INTERNAL_SERVER_ERROR | Unexpected server error |

---

# 14. Validation Requirements

Validation shall be implemented using FluentValidation or equivalent.

## 14.1 Create User Validation

- fullName required
- fullName maximum length 200
- email required
- email must be valid
- email maximum length 320

---

## 14.2 Create Wallet Validation

- userId required
- currency required
- currency length must be 3
- currency must be uppercase ISO-style code

---

## 14.3 Credit/Debit Validation

- amount required
- amount greater than 0
- currency required
- description max length 500
- idempotency key required
- metadata must be valid object

---

## 14.4 Refund Validation

- transactionId required
- amount greater than 0
- currency required

- reason required
- idempotency key required

---

# 15. Transaction Processing Flows

## 15.1 Credit Flow

1. Receive credit request.
2. Validate request body.
3. Validate idempotency key.
4. Check if same idempotency request already exists.
5. If same request exists, return stored response.
6. If same key with different request exists, return conflict.
7. Begin database transaction.
8. Lock target wallet row.
9. Validate wallet exists and is active.
10. Validate currency matches.
11. Load treasury wallet.
12. Increase user wallet balance.
13. Create transaction record.
14. Create treasury debit ledger entry.
15. Create user wallet credit ledger entry.
16. Store idempotency response.
17. Commit transaction.
18. Return success response.

---

## 15.2 Debit Flow

1. Receive debit request.
2. Validate request body.
3. Validate idempotency key.
4. Check idempotency state.
5. Begin database transaction.
6. Lock target wallet row.
7. Validate wallet exists and is active.
8. Validate currency matches.
9. Validate sufficient balance.
10. Load treasury wallet.
11. Decrease user wallet balance.
12. Create transaction record.
13. Create user wallet debit ledger entry.
14. Create treasury credit ledger entry.
15. Store idempotency response.
16. Commit transaction.

17. Return success response.

---

## 15.3 Refund Flow

1. Receive refund request.
2. Validate request body.
3. Validate idempotency key.
4. Check idempotency state.
5. Begin database transaction.
6. Load original transaction.
7. Validate original transaction is refundable.
8. Validate amount does not exceed refundable amount.
9. Lock affected wallet rows.
10. Reverse money movement.
11. Create refund transaction.
12. Create refund ledger entries.
13. Update original transaction status if fully refunded.
14. Store idempotency response.
15. Commit transaction.
16. Return success response.

---

# 16. Scalability and Future-Proofing Design

## 16.1 Module Boundaries

The system shall be organized so that the following modules can evolve independently:

- User Module
- Wallet Module
- Transaction Module
- Ledger Module
- Idempotency Module
- Reconciliation Module
- Audit Module
- Notification/Webhook Module
- Settlement Module

---

## 16.2 Interface-Based Design

Infrastructure dependencies shall be hidden behind interfaces.

Examples:

```
IWalletRepository
ITransactionRepository
ILedgerRepository
IIdempotencyStore
IUnitOfWork
IDateTimeProvider
ICorrelationIdProvider
IEventPublisher
```

This will allow future replacement of:

- PostgreSQL idempotency with Redis
- direct event publishing with outbox pattern
- simple logging with OpenTelemetry
- monolithic database with module-specific persistence

---

## 16.3 Outbox-Ready Design

The initial version does not need to publish events, but the design should support future events such as:

- WalletCreated
- WalletCredited
- WalletDebited
- TransactionPosted
- TransactionRefunded
- LedgerEntryCreated

Future table:

```
outbox_messages
---------------
id UUID PRIMARY KEY
event_type VARCHAR(200)
payload JSONB
status VARCHAR(50)
created_at TIMESTAMPTZ
processed_at TIMESTAMPTZ NULL
retry_count INT
```

---

## 16.4 Redis-Ready Design

Initial idempotency shall use PostgreSQL.

Future Redis usage may include:

- idempotency cache
- distributed locking
- read cache for balance
- rate limiting
- session/token blacklist

The application layer should not directly depend on Redis.

---

## 16.5 Authentication-Ready Design

Initial version may be unauthenticated for development simplicity.

Future authentication should support:

- JWT access tokens
- admin roles
- customer roles
- service-to-service API keys
- policy-based authorization

Controllers should be structured so authorization attributes can be added later without redesign.

---

# 17. Deployment Requirements

### 17.1 Local Development

The system shall support local startup using Docker Compose.

Required services:

- API service
- PostgreSQL database

Optional future services:

- Redis
- Kafka/RabbitMQ
- Prometheus
- Grafana

---

### 17.2 Environment Configuration

Configuration shall be environment-based.

Examples:

```
ConnectionStrings__DefaultConnection
ASPNETCORE_ENVIRONMENT
Serilog__MinimumLevel
Idempotency__ExpiryHours
TreasuryWallet__WalletId
```

Secrets shall not be committed to source control.

---

## 18. Testing Requirements

## 18.1 Unit Tests

Unit tests shall cover:

1. amount validation
2. wallet status rules
3. insufficient balance logic
4. credit domain rules
5. debit domain rules
6. refund domain rules
7. ledger balancing rules
8. idempotency conflict detection
9. transaction status transitions

---

## 18.2 Integration Tests

Integration tests shall cover:

1. create user
2. create wallet
3. get wallet balance
4. credit wallet
5. debit wallet
6. debit with insufficient balance
7. refund transaction
8. duplicate credit with same idempotency key

9. same idempotency key with different request
10. transaction history pagination
11. transaction details with ledger entries
12. concurrent debit requests
13. ledger consistency after credit
14. ledger consistency after debit
15. ledger consistency after refund

---

## 18.3 API Tests

API tests shall verify:

- correct HTTP status codes
- correct response schema
- validation errors
- business errors
- success responses
- correlation ID presence
- idempotency behavior

---

## 18.4 Suggested Test Tools

- xUnit
- FluentAssertions
- Testcontainers for PostgreSQL
- WebApplicationFactory
- Bogus for test data
- Respawn for database cleanup

---

# 19. Acceptance Criteria

The project shall be considered complete for Version 1 when:

1. .NET 10 Web API solution is created.
2. Clean Architecture structure is implemented.
3. PostgreSQL database is configured.
4. EF Core migrations are available.
5. User creation API works.
6. Wallet creation API works.
7. Balance API works.
8. Credit API works.
9. Debit API works.

10. Refund API works.
11. Transaction history API works with pagination.
12. Transaction details API returns ledger entries.
13. Every posted transaction creates ledger entries.
14. Ledger debit and credit totals match.
15. Debit cannot create negative balance.
16. Idempotency prevents duplicate money movement.
17. Same idempotency key with different request returns conflict.
18. Global exception middleware is implemented.
19. FluentValidation is implemented.
20. Serilog structured logging is implemented.
21. Correlation ID is included in logs and responses.
22. Docker Compose starts API and PostgreSQL.
23. Swagger is available.
24. Unit tests pass.
25. Integration tests pass.
26. README explains setup, architecture, APIs, and testing.

---

# 20. Development Phases

## Phase 1: Foundation

Deliverables:

- solution structure
- Clean Architecture projects
- PostgreSQL setup
- EF Core setup
- base entities
- migrations
- Docker Compose
- Swagger
- health check

---

## Phase 2: Core Wallet Ledger

Deliverables:

- user APIs
- wallet APIs
- credit API
- debit API
- transaction table
- ledger entries

- treasury wallet seed
- balance update logic
- database transactions

---

## Phase 3: Safety and Production Practices

Deliverables:

- idempotency table
- idempotency middleware/service
- request hash validation
- global exception middleware
- standard API response
- FluentValidation
- Serilog
- correlation ID
- concurrency-safe debit/credit

---

## Phase 4: Refund and Auditability

Deliverables:

- refund API
- transaction reference handling
- refund ledger entries
- transaction details API
- wallet transaction history
- ledger consistency verification

---

## Phase 5: Testing and Documentation

Deliverables:

- unit tests
- integration tests
- concurrency tests
- idempotency tests
- Postman collection
- README
- architecture diagram
- sample API requests

---

## Phase 6: Future Advanced Enhancements

Possible future deliverables:

- JWT authentication
- admin APIs
- reconciliation endpoint
- audit logs
- outbox pattern
- Redis idempotency cache
- event publishing
- rate limiting
- OpenTelemetry
- GitHub Actions CI/CD
- Kubernetes manifests

---

# 21. Recommended Version 1 Feature Set

Version 1 should include:

```
Users
Wallets
Credit
Debit
Refund
Balance
Transactions
Ledger Entries
Idempotency
Pagination
Validation
Exception Handling
Logging
Correlation ID
Docker
Swagger
Tests
README
```

Version 1 should not include:

```
Kafka
Redis
```

```
Kubernetes
Authentication
Multi-currency conversion
Partial refunds
Settlement
Webhook delivery
Admin dashboard
```

These can be added after the core financial correctness is stable.

---

## 22. Resume-Level Project Positioning

Recommended resume description:

```
Payment Ledger Service — .NET 10, ASP.NET Core, PostgreSQL, EF Core

Built a production-style wallet ledger backend supporting wallet creation,
credit, debit, refund, balance lookup, transaction history, and immutable
double-entry ledger posting. Implemented ACID-safe database transactions,
idempotency handling, concurrency-safe balance updates, structured logging,
validation, Dockerized local setup, and automated unit/integration tests.
```

---

## 23. Final System Quality Goals

The project should demonstrate the following backend engineering qualities:

1. Correctness
   Money movement should be accurate.

2. Consistency
   Wallet balance, transactions, and ledger entries should always match.

3. Idempotency
   Retried requests should not duplicate financial effects.

4. Auditability
   Every balance change should be traceable.

5. Scalability
   The system should allow future modules without major redesign.

6. Maintainability
   Code should be clean, modular, and testable.

7. Operability
   Logs, health checks, Docker, and documentation should exist.

8. Interview Readiness
   The project should be explainable through real backend concepts: transactions, concurrency, idempotency, double-entry ledger, clean architecture, and testing.